NoirFlow Expense Tracker

Database Visualization Report

Generated: December 24, 2025 at 04:07 AM

Executive Summary

This report provides a comprehensive visualization of the NoirFlow Expense Tracker database structure and data analysis. The application currently uses in-memory mock data stored in the Dashboard component (components/Dashboard.tsx). The database consists of 5 primary data models with a total of 23 records across transactions, subscriptions, monthly data, category data, and dashboard statistics.

Key Metrics

Metric	Value
Storage Type	In-Memory (Mock Data)
Location	components/Dashboard.tsx
Total Records	23
Data Models	5
Transactions	5
Subscriptions	3
Monthly Data Points	6
Category Data Points	4

Database Schema & Structure

The NoirFlow application uses a TypeScript-based data structure with the following models:

Model	Fields	Description
Transaction	6 fields	Stores expense transactions with amount, category, subcategory, date, and payment
Subscription	5 fields	Manages recurring subscriptions with name, renewal date, amount, and icon
ChartDataPoint	4 fields	Generic data structure for chart visualizations (monthly & category data)
DashboardStats	5 fields	Aggregated financial statistics including balance, expenses, investments, and goals
Enums	2 types	PaymentMode (UPI/Card/Bank/Cash) and TimeRange (Weekly/Monthly/Yearly)

Complete Database Visualization

NoirFlow Expense Tracker - Database Visualization

Database Schema & Relationships

Transaction
- id: string (PK)
- amount: number
- category: string
- subCategory: string
- date: string
- mode: enum

Subscription
- id: string (PK)
- name: string
- date: string
- amount: number
- icon: string (URL)

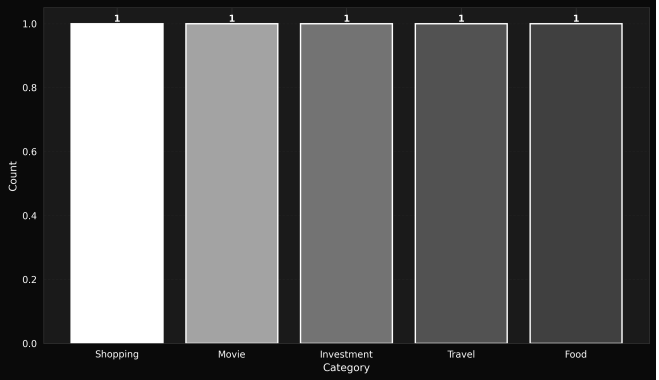
DashboardStats
- balance: number
- monthlyExpenses: number
- investment: number
- goal: number
- goalTarget: number

ChartDataPoint
- name: string
- value: number
- color?: string
- amt?: number

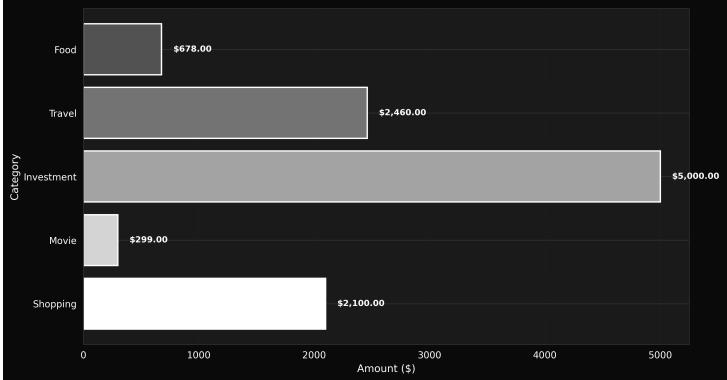
Used for: Monthly & Category

Enums & Types
- PaymentMode: UPI | Card | Bank | Cash
- TimeRange: Weekly | Monthly | Yearly
- Storage: In-Memory (Mock Data)
- Location: components/Dashboard.tsx

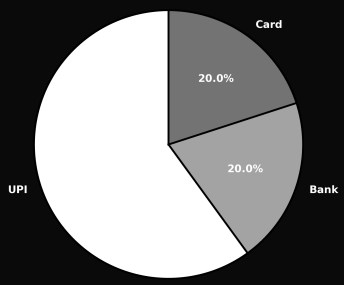
Transaction Count by Category



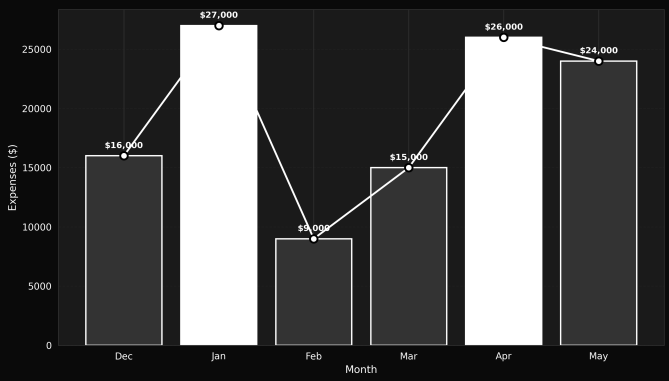
Transaction Amounts by Category



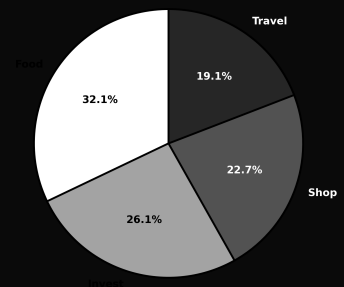
Payment Mode Distribution



Monthly Expenses Trend



Top Category Spending Distribution



Dashboard Statistics Overview

Account Balance	$898,450.00
Monthly Expenses	$24,093.00
Total Investment	$145,555.00
Goal Progress	$75,000.00
Goal Target	$145,000.00

Goal Progress: 51.7%

Active Subscriptions

Ne	**Netflix** Next: 15 June 2025	$149.00
Sp	**Spotify** Next: 24 Aug 2025	$49.00
Fi	**Figma** Next: 01 Jan 2026	$3999.00

Total Monthly Subscriptions: $4197.00

Detailed Data Analysis

The following visualizations provide in-depth analysis of the transaction patterns, spending distribution, payment preferences, and financial metrics tracked by the application.

Key Insights

• Total number of transactions recorded: 5

• Average transaction amount: $2,107.40

• Highest single transaction: $5,000.00

• Most frequently used payment method: UPI (3 transactions)

• Active subscriptions: 3 services

• Total monthly subscription cost: $4197.00

• Highest monthly expense recorded: $27,000

- Average monthly expenses: $19,500
- Goal completion progress: 51.7%
- Top spending category: Food ($6,156.00)
- Current account balance: $898,450.00
- Total investments: $145,555.00

Technical Implementation Details

Storage Architecture:
The application currently uses in-memory data storage with mock data defined directly in the Dashboard component. This approach is suitable for prototyping and demonstration purposes.

Data Location:
• Primary data source: components/Dashboard.tsx
• Type definitions: types.ts
• Service layer: services/geminiService.ts (AI insights)

Technology Stack:
• Frontend: React 19.2.1 with TypeScript
• Build Tool: Vite 6.2.0
• Charts: Recharts 3.5.1
• AI Integration: Google Gemini AI (@google/genai 1.32.0)
• Icons: Lucide React 0.556.0

Data Models (TypeScript Interfaces):
• Transaction: Expense tracking with categorization
• Subscription: Recurring payment management
• ChartDataPoint: Visualization data structure
• DashboardStats: Aggregated financial metrics
• TimeRange: Enum for time-based filtering

Future Considerations:
For production deployment, consider implementing:
• Persistent storage (LocalStorage, IndexedDB, or backend database)
• API integration for real-time data synchronization
• User authentication and multi-user support
• Data backup and export functionality
• Enhanced data validation and error handling

Recommendations

1. Database Migration: Consider migrating from in-memory storage to a persistent solution like Firebase, Supabase, or a traditional SQL/NoSQL database for production use.

2. Data Validation: Implement comprehensive input validation and sanitization to ensure data integrity across all models.

3. API Layer: Create a dedicated API service layer to abstract data operations and enable easier migration to different storage solutions.

4. State Management: Consider implementing Redux, Zustand, or React Context for more sophisticated state management as the application scales.

5. Data Export: Add functionality to export financial data in standard formats (CSV, JSON, PDF) for user records and tax purposes.

6. Real-time Sync: Implement real-time data synchronization for multi-device access and automatic backup functionality.

Report generated on December 24, 2025 at 04:07 AM
NoirFlow Expense Tracker - Database Visualization Report